

Tema V. Exemples de Llenguatges de Programació: LISP – Funcions de dibuix

21721 – Llenguatges de Programació

Antoni Oliver

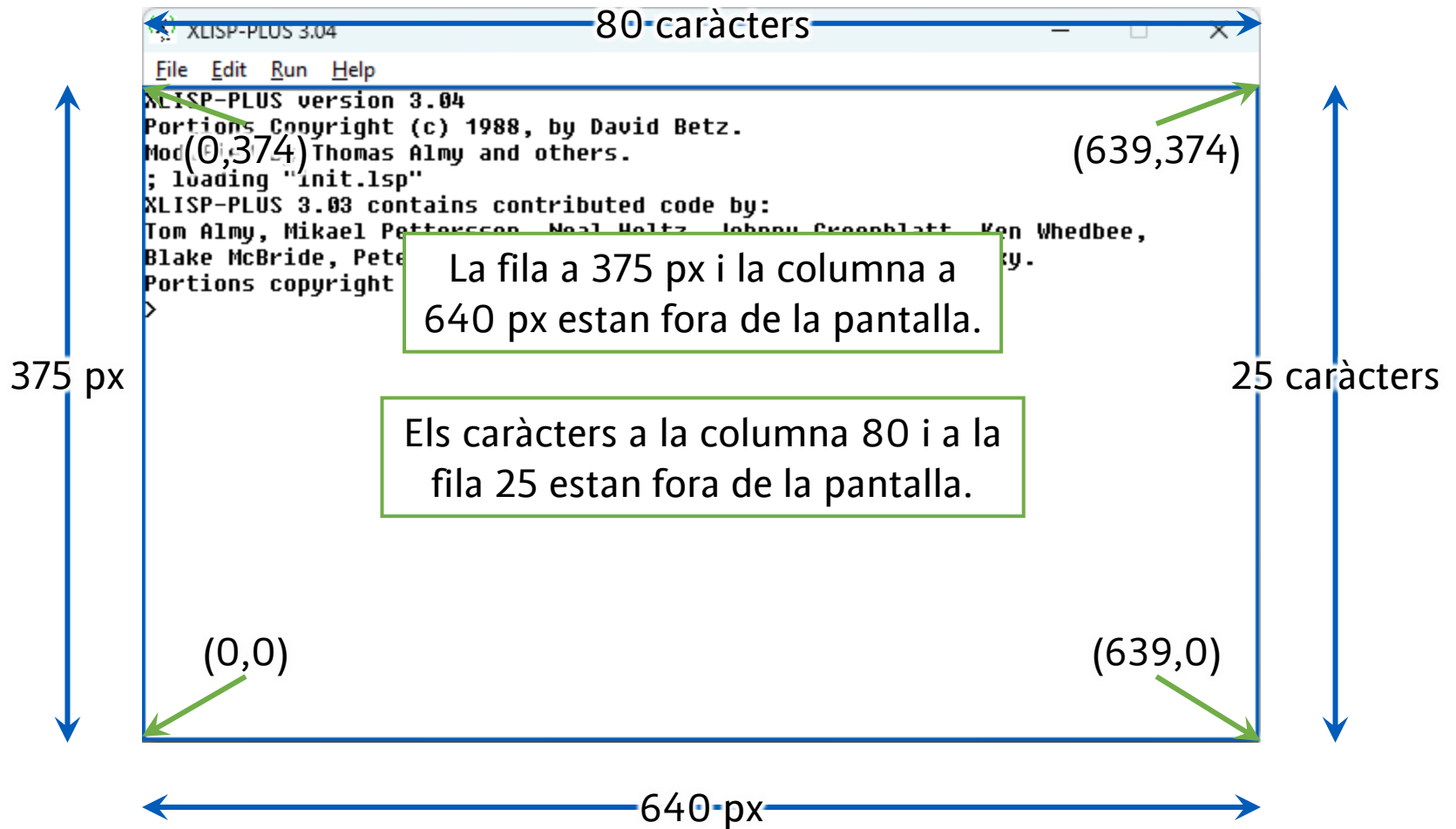
Aina M. Tur

Miquel A. Cabot

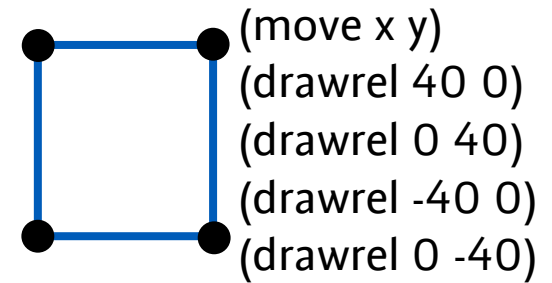
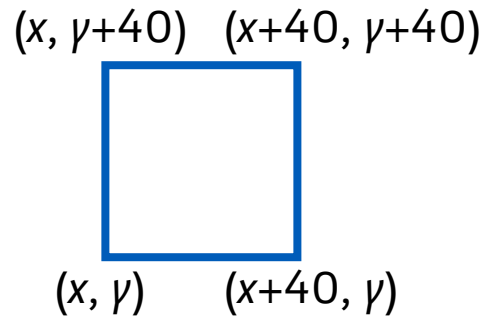


Universitat
de les Illes Balears

La finestra de dibuix d'XLISP-PLUS



Funcions de dibuix



La mida d'aquest quadrat és 41 px.

- ◆ **(move x y):**
 - ◇ Mou el cursor a la coordenada (x, y) .
- ◆ **(moverel x y):**
 - ◇ Mou el cursor x px cap a la dreta i y px cap a dalt.
- ◆ **(draw x y):**
 - ◇ Mentre mou el cursor a la coordenada (x, y) , pinta el recorregut.
- ◆ **(drawrel x y):**
 - ◇ Mentre mou el cursor x px cap a la dreta i y px cap a dalt, pinta el recorregut.
- ◆ **(color f_r f_g f_b):**
 - ◇ Estableix el color de pintat (f_i), per RGB (0-255).
- ◆ **(color f_r f_g f_b b_r b_g b_b):**
 - ◇ Estableix el color de pintat (f_i) i el de fons (b_i), per RGB (0-255).

Altres funcions relacionades amb la CLI

- ◆ **(cls):**
 - ◇ Esborra la pantalla i posa el cursor al cantó superior esquerre.
- ◆ **(read):**
 - ◇ Llegeix la línia introduïda a la CLI.
- ◆ **(get-key):**
 - ◇ Escolta la pulsació d'una tecla.
- ◆ **(goto-xy x y):**
 - ◇ Mou el cursor a la posició de caràcters indicada.
- ◆ **(cleol):**
 - ◇ Esborra la línia de caràcters des de la posició actual fins al final.

```
(defun text ()  
  (goto-xy 0 0)  
  (color 0 0 0)  
  (princ "casa")  
  (goto-xy 2 0)  
  (cleol)  
  (color 255 0 0)  
  (princ "p"))
```

capa

*Sense
cleol*

cap

*Amb
cleol*

Diferències Windows i Unix

- ◆ Amb Unix, abans hem d'obrir la finestra:
 - ◇ (mode 0 0 640 375)
- ◆ Amb Windows, és per defecte color negre sobre fons blanc.
 - ◇ Per fer-ho igual: (color 0 0 0 255 255 255).
- ◆ Amb Windows, el cursor es manté entre crides del *top level*.
 - ◇ Això només afecta fer crides individuals amb la *REPL*.
 - ◇ Tanmateix, acabarem agrupant-les en funcions com *quadrat*.
- ◆ Amb Windows, només podem posicionar text amb *goto-xy*, *cleol* funciona correctament.
 - ◇ Amb Unix, també podem posicionar-lo amb *move* i *moverel*, però *cleol* pareix que no funciona correctament.
- ◆ Amb Windows, dibuixar és lent.
 - ◇ Amb Unix, és molt ràpid. També tenim més funcions disponibles (qt.lsp).

Funcions de dibuix

◆ Funció quadrat que pinta un *quadrat* de mida m .

◇ (quadrat 40)

```
(defun quadrat (m)
  (drawrel m 0)
  (drawrel 0 m)
  (drawrel (- m) 0)
  (drawrel 0 (- m)))
```

Funcions de dibuix

◆ Què fa aquesta funció?

◇ `(quadrats 3 50 50)`

```
(defun quadrats (n d1 d2)
  (cond ((= n 0) nil)
        (t (moverel d1 d2)
            (quadrat 40)
            (quadrats (- n 1) d1 d2)))))
```

◇ Repeteix n vegades:

- Desplaçar-se (d1, d2)
- Pintar un quadrat de mida 40

Funcions de dibuix

◆ Què fa aquesta funció?

◆ (m)

```
(defun m ()  
  (cls)  
  (move 150 100)  
  (quadrats 20 4 4)  
  (quadrats 20 4 -4)  
  (quadrats 20 4 4)  
  (quadrats 20 4 -4))
```

◆ Esborra la pantalla

◆ Es mou a (150, 100)

◆ 20 quadrats ↗

◆ 20 quadrats ↘

◆ 20 quadrats ↗

◆ 20 quadrats ↘

```
(defun cercle (x y radi segments)
  (mover (+ x radi) y)
  (cercle2 x y radi (/ 360 segments) 0))
```

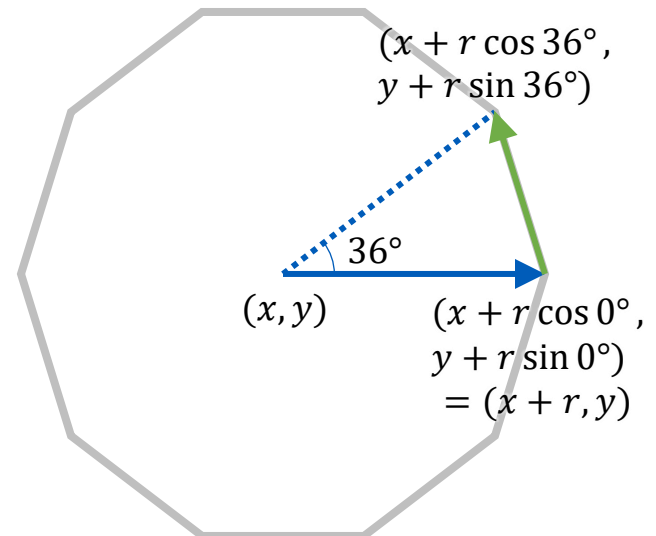
```
(defun cercle2 (x y radi pas angle)
  (cond ((< angle 360)
    (drawr (+ x (* radi (cos (radians (+ angle pas))))))
    (+ y (* radi (sin (radians (+ angle pas))))))
    (cercle2 x y radi pas (+ angle pas)))
  (t t)))
```

```
(defun mover (x y)
  (move (round x)
    (round y)))
```

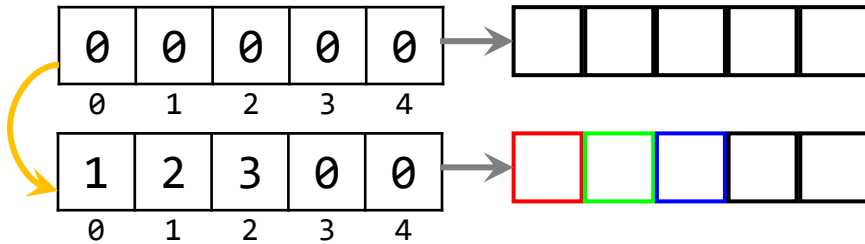
```
(defun drawr (x y)
  (draw (round x)
    (round y)))
```

```
(defun radians (graus)
  (/ (* graus (* 2 pi)) 360))
```

◇ (cercle 200 200 50 10)



Problema gràfic



0	(0 0 0)
1	(255 0 0)
2	(0 255 0)
3	(0 0 255)

◆ Preparem algunes constants:

◇ (setq xi 10) (setq yi 10)
(setq m 25)

◆ Llista de cinc zeros:

◇ (setq l (list 0 0 0 0 0))

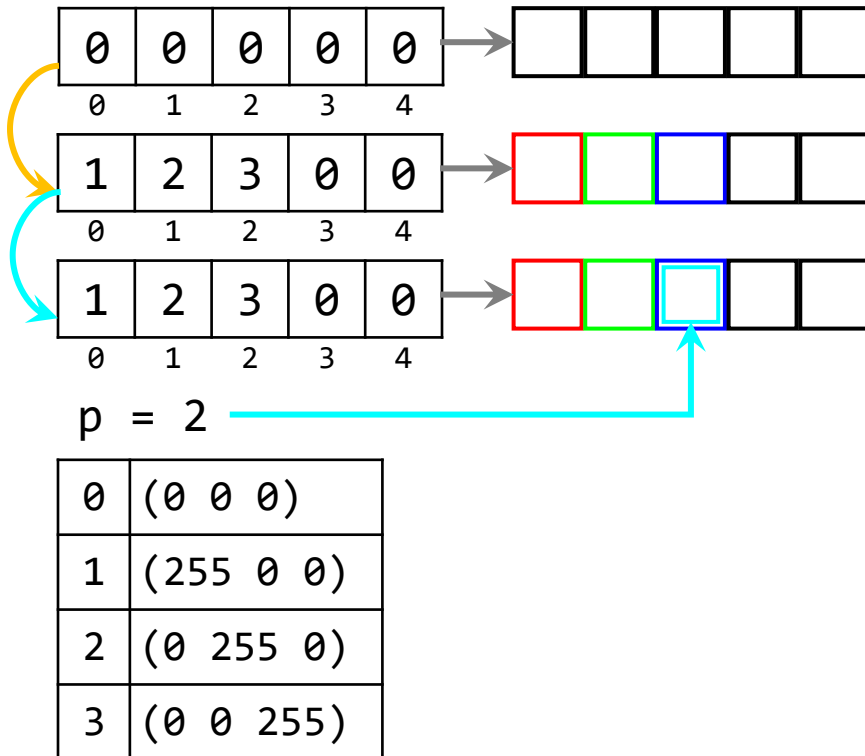
◆ Funció per **canviar** el valor a l'índex *i* de la llista *L* per *v*: (canvia l i v)

◇ Avalua a una altra llista, amb el valor canviat.

◆ Funció que **pinta** una llista *L*: (pinta l)

- ◇ a partir d'una coordenada inicial (x_i, y_i),
- ◇ pinta, per cada element, un quadrat de mida $m-1$ px, separats m px,
- ◇ amb els costats del color segons el valor.

Problema gràfic



◆ Posició actual, inicialment 2:

◇ (setq p 2)

◆ Funció **dreta**, que avalua a la posició de la dreta de p :
(dreta p)

◇ El mateix amb (esquerra p).

◇ Avalua a una altra posició, amb el valor canviat.

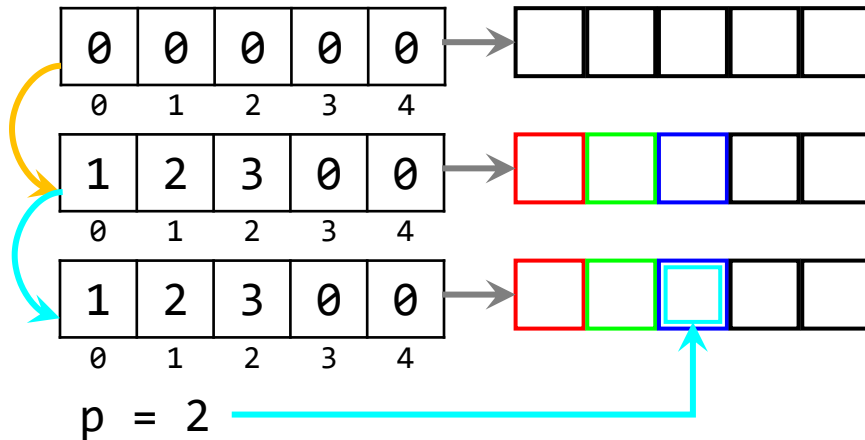
◇ Mòdul 5: (mod n 5).

◆ Modifica **pinta**, perquè accepti p :
(pinta l p)

◇ En pintar l'element p , pinta un altre quadrat d' $m-5$ px a dins, de color cyan (0 255 255).

◆ Esborra les variables L i p .

Problema gràfic



0	(0 0 0)
1	(255 0 0)
2	(0 255 0)
3	(0 0 255)

Tecles		Valors numèrics	
A	a	65	97
D	d	68	100
0		48	
1		49	
2		50	
3		51	
ESC		27	

(every 'plusp llista)

◆ Funció **passa**, que pren una llista i una posició:

(passa '(0 0 0 0 0) 2)

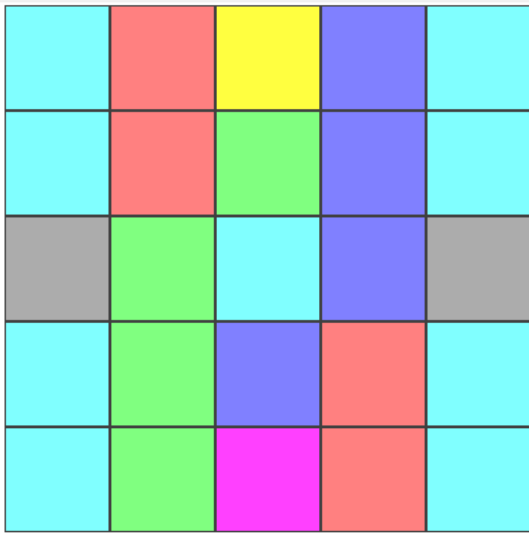
- ◆ Pinta la llista i la posició.
- ◆ Si tots (every) els elements tenen color (>0), avalua a la llista (acaba la recursivitat).
- ◆ Si no, escolta la tecla:

- **A/D**: es crida recursivament amb la llista original i la posició resultant de fer servir *esquerra* o *dreta*.
- **0/1/2/3**: es crida recursivament amb la llista resultant de fer servir *canvia* amb la llista original, la posició (original) i la tecla pitjada, i la posició original.
- **ESC**: avalua a la llista (acaba la recursivitat).
- **Altres**: es crida recursivament tal qual.

Pràctica final

◆ Aquesta idea és part de la idea de la pràctica final.

```
(  
  ((aigua)      (terra r) (terra g base e2) (terra b) (aigua)      )  
  ((aigua)      (terra r) (terra g)          (terra b) (aigua)      )  
  ((terra r lab) (terra g) (aigua)            (terra b) (terra r lab))  
  ((aigua)      (terra g) (terra b)          (terra r) (aigua)      )  
  ((aigua)      (terra g) (terra b base e1) (terra r) (aigua)      )  
)
```



◇ Estat = matriu 2D = llista de files:

- Cada fila = llista de caselles:
 - Cada casella = llista de què hi ha:
 1. tipus (aigua / terra)
 2. color (r / g / b)
 3. element (*nil* / base / lab / bolla)
 4. equip (e1 / e2)
 5. més...

Pràctica final

◆ Per llegir l'expressió del fitxer del mapa:

- ◇ `(llegeix-exp "maps/cincpercinc.map")`

```
(defun llegeix-exp (nom-fitxer)
  (let* ((fp (open nom-fitxer))
        (e (read fp nil nil)))
    (close fp)
    e))
```

◆ Adaptar el que hem fet (part de pintar):

- ◇ Llista 1D -> 2D. Diferents valors per casella.
- ◇ Les unitats també apareixen al mapa (bases i bolles).
- ◇ Només pintar els canvis en comptes de repintar-ho tot?

◆ Controlador: processa torns, alternant equips (e1 / e2).

- ◇ Processa la llista d'unitats per equip.
 - Cada unitat tria accions segons la info que li arriba (funció IA).
 - Se genera un estat nou amb aquestes accions.